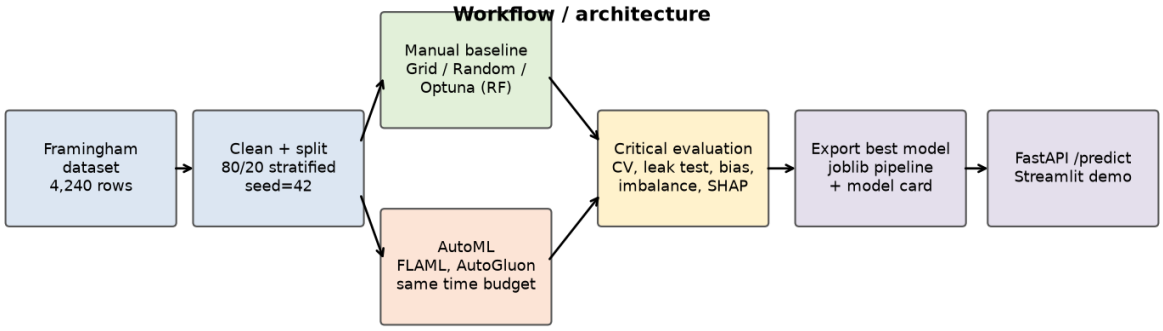
AutoML Skill-Based Project (IA-3)

Manual vs AutoML for 10-year Heart Disease Risk Prediction

Subject: AutoML

Team members: [add names and register numbers]

Dataset: Framingham Heart Study (4,240 rows) | Random seed: 42 | AutoML budget per tool: 240 s

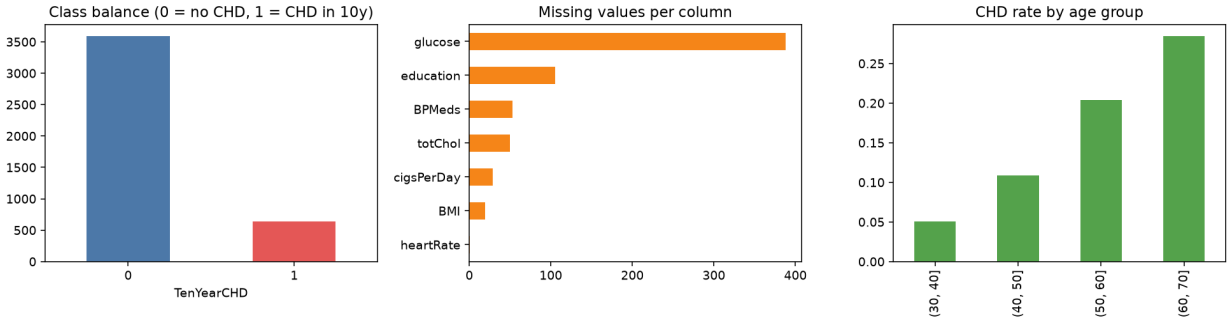


1. Problem statement and dataset

Problem. Cardiovascular disease is a leading cause of death. Given routine clinical measurements, can we estimate whether a person will develop coronary heart disease (CHD) within ten years? This is a binary classification task with an imbalanced target. We use it to compare a carefully engineered manual scikit-learn workflow against two AutoML frameworks and to audit all of them for leakage, bias and explainability.

Dataset. Public Framingham Heart Study extract (4,240 patients, 15 predictors, target *TenYearCHD*). 15.2% of patients are positive. 645 cells are missing (glucose 388, education 105, BPMeds 53, totChol 50, cigsPerDay 29, BMI 19, heartRate 1). There are 0 duplicate rows.

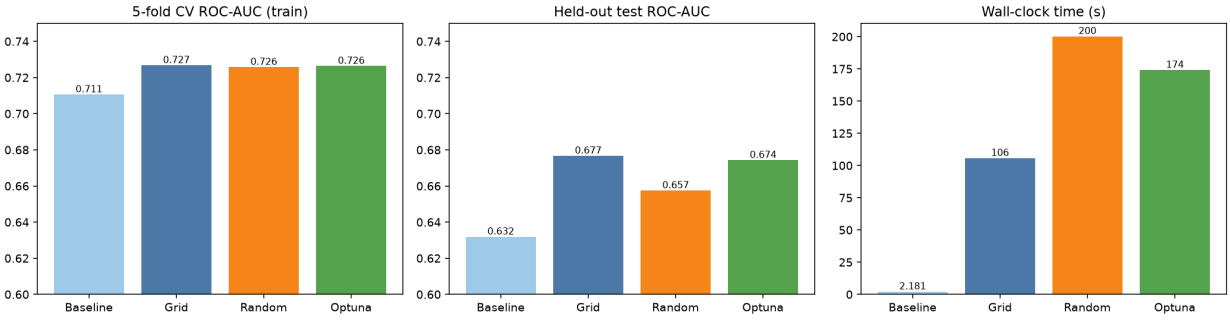
Validation design. One stratified 80/20 split (seed 42). All tuning, AutoML search and threshold selection use only the 80% training part (5-fold stratified CV). The 20% test set is scored once per model at the end. Imputation is inside the sklearn pipeline, so it is re-fitted in every CV fold (no preprocessing leakage).



2. Manual baseline and hyper-parameter tuning

The baseline is a default RandomForest in a Pipeline (median imputer + forest). The **same model** was then tuned with Grid Search, Random Search and Bayesian optimisation (Optuna TPE). Every tuner got exactly 36 evaluations with 5-fold CV on ROC-AUC, so time differences reflect the method, not the budget. ROC-AUC is the main metric because with 15% positives accuracy (≈85% for predicting all negatives) and F1 at 0.5 are misleading.

Method	CV AUC	CV std	Test AUC	Test F1@0.5	Time (s)	Evals
Dummy (prior)	0.5	0.0	0.5	0.0	0.101	5
Logistic Regression	0.731	0.03	0.702	0.096	0.166	5
RandomForest (default) = BASELINE	0.711	0.033	0.632	0.071	2.181	5
Grid Search	0.727	0.035	0.677	0.03	105.764	36
Random Search	0.726	0.034	0.658	0.071	200.3	36
Optuna (Bayesian/TPE)	0.726	-	0.674	0.03	174.104	36



Observations.

- All three tuners reach almost the same CV AUC (0.727 / 0.726 / 0.726); the differences are far smaller than the CV standard deviation (≈0.03). Tuning improved the baseline CV AUC by about 0.016.
- Grid search was fastest here (106 s) only because our grid is small and contains cheap (shallow, few-tree) settings; Random (200 s) and Optuna (174 s) sample bigger forests (up to 500 trees). Time per method therefore depends on the search space as well as the algorithm.
- A plain scaled Logistic Regression is competitive (test AUC 0.702) and takes 0.2 s. Signal in this dataset is mostly linear, so complex models add little.
- Best parameters: Grid {"max_depth": 8, "max_features": "sqrt", "min_samples_leaf": 10, "n_estimators": 100}; Random {"max_depth": 17, "max_features": 0.316, "min_samples_leaf": 7, "n_estimators": 108}; Optuna {"n_estimators": 424, "max_depth": 8, "min_samples_leaf": 3, "max_features": 0.236}.

3. AutoML implementation and tool comparison

Two frameworks were run on the identical training data and scored on the identical test set: **FLAML** (cost-frugal search over LightGBM, XGBoost, CatBoost, RF, ExtraTrees, linear models; 5-fold CV inside) and **AutoGluon-Tabular** (preset best_quality: many bagged models + weighted ensemble). Each tool had a fixed wall-clock budget of **240 seconds** on 2 CPU cores, metric ROC-AUC, seed 42. (The assignment suggests 15–30 min; the budget is one variable, *AUTOML_BUDGET*, and the reproduction scripts accept 900 or 1800.)

FLAML leaderboard (best 5-fold CV AUC reached by each learner; 'improving configs logged' counts configurations that improved the best score and were written to the log):

Learner	Best CV AUC	Improving configs logged
extra_tree	0.7285	2.0
catboost	0.7225	2.0
lgbm	0.7183	7.0
xgb_limitdepth	0.718	-
xgboost	0.7179	-
rf	0.7158	-
sgd	0.6801	-
lrl1	0.6144	-

FLAML selected **extra_tree** (config {"n_estimators": 125, "max_features": 0.632, "max_leaves": 64, "criterion": "gini"}).

AutoGluon leaderboard (top 8 of 49 models trained; score_val is AutoGluon's internal out-of-fold AUC, score_test is our held-out test AUC):

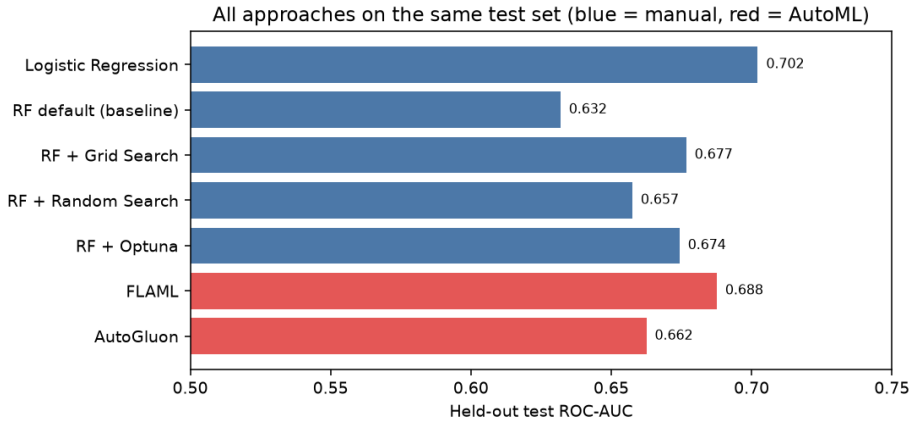
Model	Val AUC (OOF)	Test AUC	Fit time (s)	Stack level
ExtraTrees_r4_BAG_L1	0.7225	0.6901	0.8236	1
LightGBM_r96_BAG_L1	0.7334	0.6825	3.129	1
RandomForest_r34_BAG_L1	0.7151	0.6795	0.8219	1
ExtraTrees_r126_BAG_L1	0.7287	0.6794	0.9378	1
WeightedEnsemble_L2	0.7366	0.6778	7.7246	2
LightGBM_r94_BAG_L1	0.7348	0.6774	2.4384	1
LightGBMXT_BAG_L1	0.7328	0.6762	2.5697	1
ExtraTrees_r178_BAG_L1	0.7237	0.6751	1.0233	1

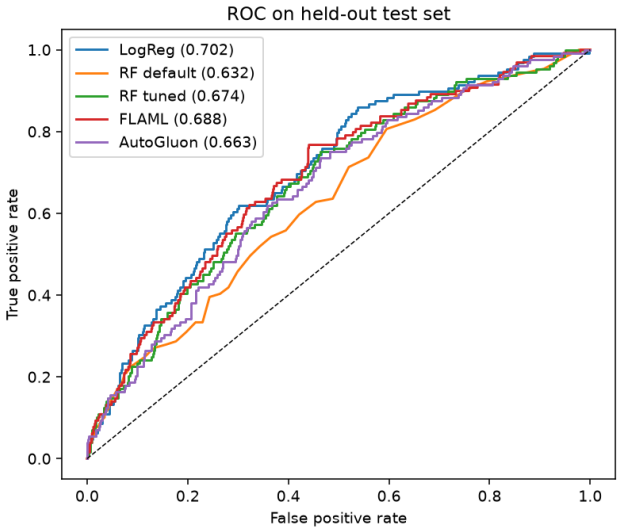
AutoGluon's winner was **WeightedEnsemble_L3**; The best single-model family on the AutoGluon leaderboard was **ExtraTrees** (families that trained successfully: ExtraTrees, ExtraTreesEntr, ExtraTreesGini, LightGBM, LightGBMLarge, LightGBMXT, RandomForest, RandomForestEntr, RandomForestGini). Model families that failed to import or train in the environment (e.g. torch/fastai neural nets, or XGBoost with the installed scikit-learn version; see results/03_automl.log) were skipped by AutoGluon and not tried (limitation).

Preprocessing / feature engineering applied by the tools. FLAML: its built-in data transformer (imputation of missing values and encoding of categoricals, per FLAML documentation); no new features. AutoGluon: AutoMLPipelineFeatureGenerator – automatic type inference, native NaN handling for tree models, removal of constant/duplicate columns. We checked *predictor.features()*: it used the same 15 input columns and created no new ones. With an all-numeric table there is little for either tool to engineer.

Final comparison on the held-out test set

Model	CV/val AUC	Test AUC	Train time (s)	F1@0.5	Test AUC 95% CI
Logistic Regression	0.7312	0.7022	0.1663	0.0959	0.657–0.751
RF default (baseline)	0.7106	0.6319	2.1812	0.0709	0.581–0.687
RF + Grid Search	0.7268	0.6767	105.7645	0.0299	-
RF + Random Search	0.7258	0.6575	200.3003	0.0714	-
RF + Optuna	0.7264	0.6743	174.1044	0.0299	0.628–0.726
FLAML (240s budget)	0.7285	0.6876	240.4168	0.0441	0.640–0.739
AutoGluon (240s budget)	0.7503	0.6625	240.389	0.0986	0.614–0.719





Interpretation.

- No approach is significantly better: all 95% bootstrap intervals overlap heavily (test set has only 129 positives). The point estimates rank Logistic Regression (0.702) > FLAML (0.688) > AutoGluon (0.662) > tuned RF (0.674) > default RF (0.632), but this ordering should not be over-interpreted.
- Both AutoML tools beat the manual untuned/tuned RF on test AUC slightly but not the simple Logistic Regression. AutoML's value here was speed of obtaining a strong, well-validated model with zero manual design, not a higher ceiling.
- Training time: baseline 2 s, logistic regression 0.1 s, manual tuning 110–215 s per method, AutoML = the full budget by design (FLAML and AutoGluon keep searching until time runs out).
- Interpretability: Logistic Regression (coefficients) > tuned RF (exact TreeSHAP) > FLAML single ExtraTrees (SHAP possible) > AutoGluon (a weighted ensemble of 29 bagged models: explainable only through

model-agnostic methods and much heavier to deploy).

4. Critical evaluation

4.1 Overfitting and validation

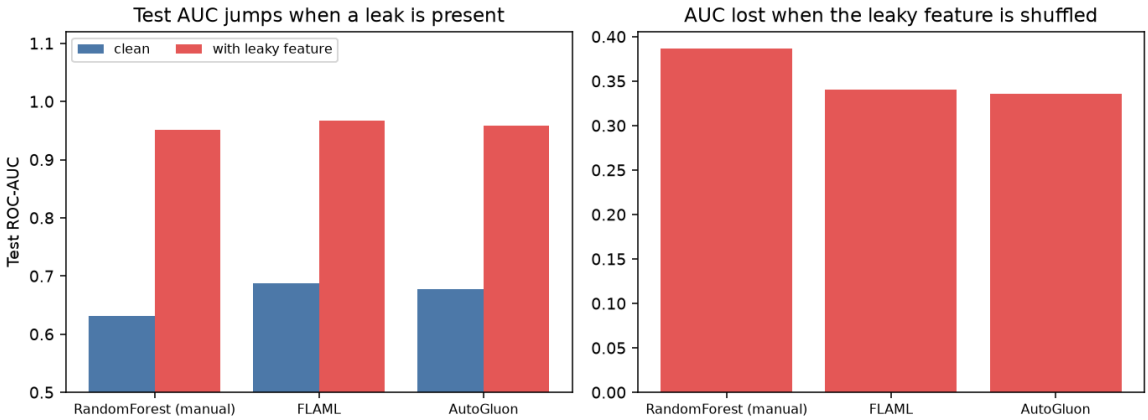
Model	Train AUC (inside CV)	CV val AUC	Train AUC (full fit)	Test AUC
RF default (baseline)	1.0	0.711	1.0	0.632
RF tuned (Optuna)	0.906	0.726	0.891	0.674

The default forest memorises the training data (train AUC 1.000 vs CV 0.711) – classic overfitting that the held-out test set confirms (test 0.632). Tuning (shallower, more regularised trees) lowers train AUC to 0.906 and raises CV and test AUC. Note the CV-to-test drop for the tuned model (0.726 → 0.674) is within what a 848-patient test set can produce (bootstrap CI ±0.05), so we report both numbers rather than only the better one.

4.2 Deliberate data leakage experiment

We injected a leaky feature *cardiac_medication_started* = the true outcome with 5% of values randomly flipped (in real life: a drug started **after** a cardiac event, i.e. information from the future). It was added to train and test. The manual forest used default settings; FLAML and AutoGluon were re-run from scratch with half the main budget (120 s).

Model	Test AUC clean	Test AUC with leak	AUC lost if leak shuffled
RandomForest (manual)	0.632	0.952	0.386
FLAML	0.688	0.966	0.34
AutoGluon	0.678	0.959	0.336



Result: every method exploits the leak immediately: test AUC jumps from ≈0.63–0.69 to ≈0.95–0.97, and shuffling the leaky column destroys 0.34–0.39 AUC. AutoGluon's own feature-importance ranks it #1. **AutoML does not detect leakage** – it optimises the metric it is given. A simple single-feature screen did expose the problem before modelling: the leak alone has AUC 0.945 while the best genuine feature (age) has 0.690. An AUC above ~0.9 on a task where the literature reports ≈0.7–0.75 is itself a red flag.

4.3 Class imbalance

Setting	CV AUC	Test AUC	PR-AUC	Recall@0.5	Precision@0.5	F1@0.5
no weighting	0.726	0.674	0.277	0.016	0.4	0.03
class_weight='balanced'	0.719	0.674	0.285	0.512	0.261	0.346

Only 15.2% of patients are positive. At the default 0.5 threshold the tuned forest finds 1.6% of true cases – accuracy (≈85%) hides this. Class weighting leaves ranking quality (AUC) unchanged but raises recall at 0.5 to 51%; alternatively a lower threshold (0.18, chosen on out-of-fold *training* predictions only) gives recall 0.48,

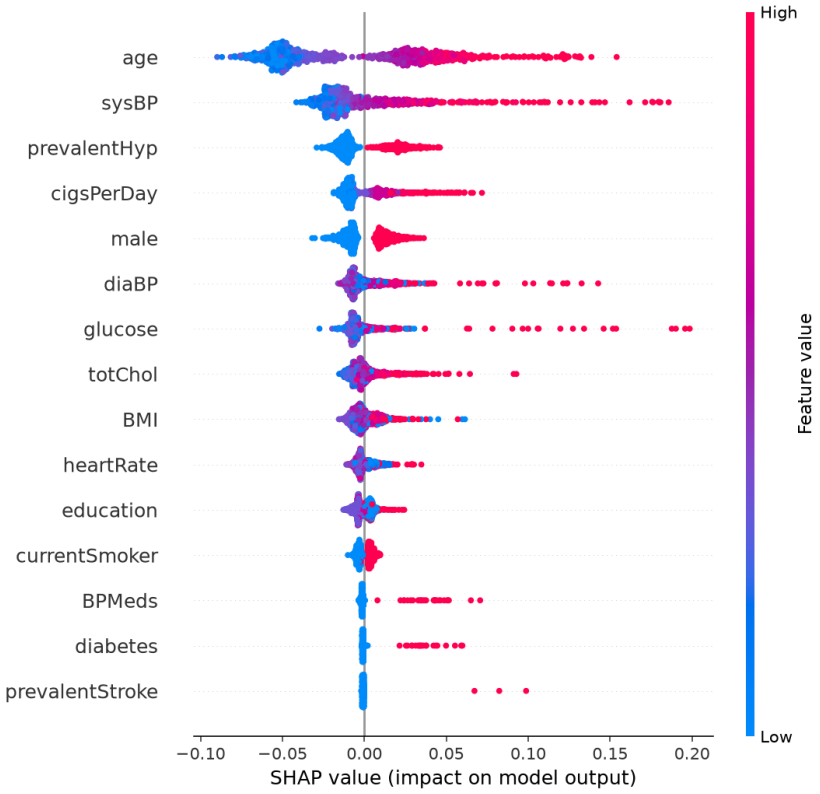
precision 0.25, F1 0.33 on the test set. The decision threshold is a clinical/cost choice, not a modelling detail.

4.4 Subgroup bias (tuned RF at threshold 0.18)

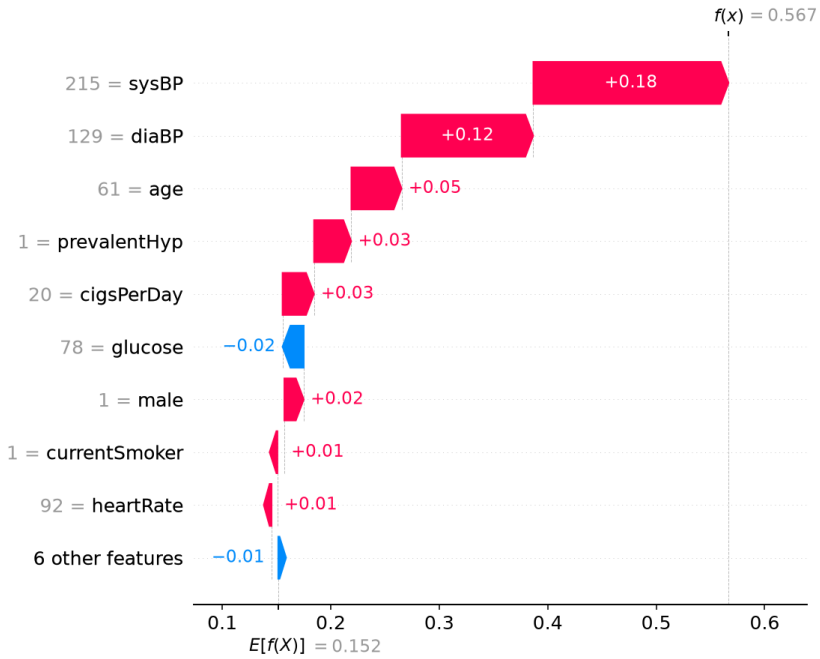
Attribute	Group	n	CHD prevalence	AUC	Flag rate	Recall (TPR)	False-pos. rate
sex	Female	482	0.108	0.659	0.237	0.404	0.216
sex	Male	366	0.21	0.66	0.369	0.532	0.325
age	<45	299	0.09	0.626	0.04	0.111	0.033
age	45-54	291	0.141	0.626	0.268	0.39	0.248
age	55+	258	0.236	0.605	0.616	0.705	0.589

Men are flagged more often than women (37% vs 24%) and have both higher recall and a higher false-positive rate; part of this reflects the real prevalence gap (21% vs 11%). Discrimination (AUC) is similar for both sexes (0.66 vs 0.66). The model is strongly age-driven: patients under 45 are almost never flagged (recall 0.11) while 62% of those 55+ are flagged. Within-group AUC is lower than overall AUC (0.63–0.60) because much of the overall ranking comes from age alone. Subgroups are small (n=258–482), so these figures are indicative, not definitive. No fairness mitigation was applied.

4.5 Explainability (SHAP)



Global SHAP (TreeExplainer on the tuned forest, test set) ranks **age** first (mean |SHAP| 0.045), then **systolic BP** (0.023), hypertension, cigarettes per day and sex. All directions are clinically plausible (older, higher BP, more cigarettes, male → higher risk). Permutation importance on the test set agrees on the top features (age drop 0.060, sysBP 0.016) and shows that several low-ranked features (totChol, BMI, glucose) contribute ≈0 or slightly negative AUC out-of-sample – evidence the forest uses them partly as noise.



Local explanation for the highest-risk test patient (also returned live by the API/app).

4.6 Limitations found

- Weak signal: best test AUC ≈0.70; clinical use would need much better discrimination and calibration.
- Small test set (848 patients, 129 positives): CV and test AUC differ by ≈0.05 and model rankings are within noise. A single split was used for the final comparison; repeated CV across all tools would be better but exceeds the time budget.
- AutoML budget used for the reported numbers was 240 s per tool on 2 cores (shorter than the 15–30 min suggested), so FLAML could evaluate only a limited number of configurations (5-fold CV on 2 cores); longer budgets may change the ranking slightly. Some AutoGluon model families (neural nets, and XGBoost in our environment) failed to run and were skipped.
- Threshold F1 optimisation and probability calibration were only lightly explored; probabilities are not guaranteed calibrated.
- Dataset is a single historical US cohort; the education variable is of questionable causal meaning and glucose has 9% missing values (median imputation).
- The leakage experiment used a synthetic leak; real leaks (e.g. duplicated patients, time leakage) are subtler.

5. Deployment

Model choice. Rule fixed before looking at test scores: deploy the Optuna-tuned sklearn pipeline unless another candidate beats its CV AUC by more than 0.01 (CV std ≈0.03). FLAML's CV AUC was 0.728 vs 0.726, a difference of +0.002, so the simpler tuned forest was exported (*model/heart_risk_pipeline.joblib*). It needs only scikit-learn, supports exact SHAP and avoids shipping AutoGluon's 29-model directory.

Serving. *app/api.py* is a FastAPI service (POST /predict with validated inputs, GET /health, GET /model-info, interactive docs at /docs). *app/streamlit_app.py* is a Streamlit demo with sliders, predicted 10-year risk vs the dataset base rate, the alert flag and a SHAP bar chart of the top factors; the model card is shown in a second tab. Both import *app/serving.py*, so they always give identical answers. Both were smoke-tested (valid prediction, 422 on invalid input).

Run locally: *uvicorn app.api:app --reload* and *streamlit run app/streamlit_app.py* (see README). The Streamlit app can also be deployed free on Streamlit Community Cloud from a GitHub repo.

The full model card is in MODEL_CARD.md (data, metrics, intended use, known limits).

6. Reflections: what AutoML did well and where it failed

Did well

- Produced a competitive, properly cross-validated model in a few minutes with no manual feature or model design; FLAML's CV AUC (0.728) matched our 36-trial manual search (0.726).
- Transparent leaderboards showed which families work (FLAML: extra_tree on top; AutoGluon: ExtraTrees on top; linear models weakest in CV).
- Handled missing values and types automatically.

Failed / needs a human

- Did not beat a one-line logistic regression on test AUC, and cannot raise the information ceiling of weak data.
- Exploited the leaky feature instantly and gave no warning – validation scores looked superb. A human must audit features.
- Default decision threshold and class imbalance were not handled in a clinically meaningful way (recall ≈2–3% at 0.5 for both tools).
- Heavy ensembles (29 models) are hard to explain and deploy; the 'best' model is chosen on noisy validation scores.
- Results depend strongly on the time budget and hardware, so tool comparisons must fix both (we did).

Takeaway: AutoML is an excellent baseline generator and productivity tool, but validation design, leakage hunting, bias analysis and threshold choice remain human responsibilities.

7. Reproducibility

All code is in *src/* and runs in order 01 → 07. Fixed seed 42; one shared split in *src/common.py*. Set *AUTOML_BUDGET=900* (or 1800) to reproduce with the longer budgets recommended by the brief. Wall-clock-based AutoML search is not exactly deterministic across machines, so expect small differences in AutoML numbers.